Full-Stack Quest

Coordination développement Front et Back

Négocier l'interface

Git, monorepo, premier CI

Rejoindre la classe



localhost:5173/rejoindre

Date

Vendredi 30 octobre 2026

Horaires · salle

9 h – 12 h · S102

Chapitre

03 / 13

Durée

3 h

Jeu du jour

API Diplomacy N1-3

Mastère 2 Expert en développement Full Stack · A2627_0195

Le déroulé du jour

Vendredi 30 octobre 2026 · 9 h – 12 h

9 h 00		**Échauffement Contract Tetris** 15 min Pack « verbes et idempotence », 3 manches de 3 min, classement projeté. Les retardataires rejoignent ici.
9 h 15		Théorie · Git, monorepo, premier CI 35 min
9 h 50		API Diplomacy N1-3 40 min
10 h 30		Pause 10 min
10 h 40		Repo et premier CI 1 h 20

Contract Tetris

Échauffement · pack « verbes et idempotence »

01 Trois manches de 3 minutes, pack du jour.

02 Thème : quel verbe pour quelle opération, et que se passe-t-il si la requête est **rejouée** ?

03 Seules les 3 premières manches comptent pour l'XP.

Temps de jeu

15:00

▶ Cliquer pour lancer

Compétence

Lire un contrat REST d'un coup d'œil

Où jouer

localhost:5173 → hub → Contract Tetris

Git, monorepo, premier CI

Théorie · 9 h 15 · 35 min

01 Stratégies de branches

02 Monorepo ou multi-repo

03 Conventional Commits

04 Un premier CI avec GitHub Actions

Trois stratégies de branches

	GitFlow	GitHub Flow	Trunk-based
Branches	`main`, `develop`, `release/*`, `hotfix/*`, `feature/*`	`main` + branches courtes	`main` + branches de quelques heures
Intégration	à la fin de la fonctionnalité	à chaque PR	**plusieurs fois par jour**
Fonctionnalité inachevée	reste sur sa branche	reste sur sa branche	fusionnée, **cachée derrière un flag**
Fait pour	logiciel versionné, plusieurs versions maintenues	services web, petites équipes	équipes qui livrent en continu

Une branche qui vit une semaine est un contrat que personne n’a signé.

Pendant qu’elle vit, le front et le back divergent sans le savoir. Le conflit de fusion n’est que le symptôme : le vrai coût, c’est l’intégration repoussée.

Coordonner front et back dans Git

- Un changement de contrat est **une PR à part**, fusionnée **avant** le code qui l'utilise.
- Elle est relue par **les deux** sous-groupes : `CODEOWNERS` l'impose.
- Le code front et back suit dans des PR séparées, qui référencent la PR de contrat.
- Le titre de la PR suit la convention de commit : il devient le message du *squash*.

```
# .github/CODEOWNERS
/contracts/          @alice @bilal @chloe @dan
/apps/web/           @alice @bilal
/apps/api/           @chloe @dan
/.github/workflows/  @alice @dan
```

Monorepo ou multi-repo

	Monorepo	Multi-repo
Changer le contrat et ses deux côtés	une seule PR **atomique**	trois PR à synchroniser
Partager des types	un paquet du workspace	un paquet publié et versionné
CI	doit savoir ne tester **que ce qui change**	simple, un dépôt = un pipeline
Droits	les mêmes pour tous (ou `CODEOWNERS`)	fins, par dépôt
Cycles de livraison	souvent liés	indépendants

Pour votre projet : **monorepo**. Le contrat bouge souvent et doit bouger avec ses deux côtés.

Une structure qui place le contrat au centre

```
cache-racer/
├─ contracts/
│  └─ openapi.yaml          ← la source de vérité, relue par tous
├─ apps/
│  ├─ web/                  ← le front (sous-groupe front)
│  └─ api/                  ← le back (sous-groupe back)
├─ packages/
│  └─ api-types/            ← types générés depuis le contrat
├─ docs/
│  ├─ adr/                  ← une décision par fichier
│  └─ pitch.md
├─ .github/workflows/ci.yml
├─ pnpm-workspace.yaml
└─ README.md
```

- Le contrat n'appartient **ni au front ni au back**
- Les types générés sont un paquet du workspace, importé des deux côtés si TypeScript
- Un back en Python ? Il lit le même YAML : le monorepo n'impose pas le langage

Conventional Commits

```
<type>(<portée>)<!>: <description>

feat(contract): ajouter GET /api/levels/{levelId}/leaderboard
fix(web): afficher l'erreur 422 sous le champ score
feat(api)!: renommer level en levelId dans RunInput
test(api): couvrir le jeton expiré sur POST /runs
ci: lancer Spectral sur chaque PR
docs(adr): 0003 choisir TanStack Query

BREAKING CHANGE: les clients doivent envoyer levelId (chaîne).
```

- Types usuels : `feat`, `fix`, `docs`, `refactor`, `test`, `ci`, `perf`, `chore`
- Portées du projet : `contract`, `web`, `api`
- `!` ou le pied de page `BREAKING CHANGE` signalent une **rupture**
- Une rupture sur `contract` doit toujours être visible

Pourquoi s’imposer cette syntaxe

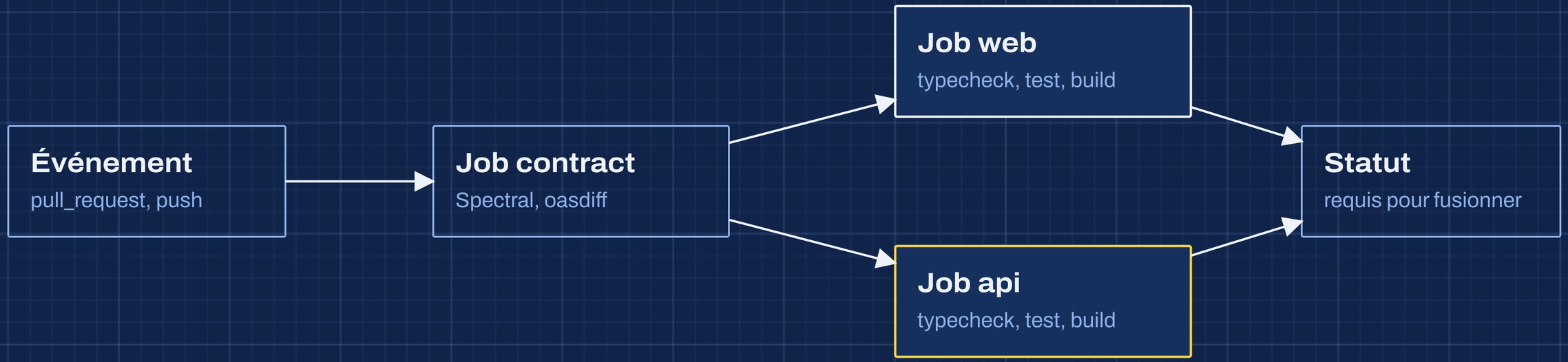
- Un **historique lisible** par l’autre sous-groupe : `git log --oneline -- contracts/`
- Un **CHANGELOG** généré au lieu d’être oublié
- Une **version** calculée : `fix` → correctif, `feat` → mineure, `!` → majeure
- Une **rupture de contrat** impossible à glisser en douce

Où l’imposer

En CI, sur le **titre de la PR**, avec la fusion en *squash* : un contrôle, un endroit, aucun hook local à installer.

Les hooks locaux (husky + commitlint) sont un confort, pas une garantie : ils se contournent avec `--no-verify`.

GitHub Actions en une slide



- Un **workflow** réagit à des événements ; il contient des **jobs** (machines isolées) faits de **steps**.
- `needs` ordonne les jobs : si le contrat est invalide, inutile de construire le reste.

`ci.yml` (1/2) : le contrat d'abord

```yaml
name: CI
on:
  pull_request:
  push: { branches: [main] }
concurrency: { group: "ci-${{ github.ref }}", cancel-in-progress: true }
jobs:
  contract:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5
      - uses: pnpm/action-setup@v4
      - uses: actions/setup-node@v5
        with: { node-version: 24, cache: pnpm }
      - run: pnpm install --frozen-lockfile
      - run: pnpm exec spectral lint contracts/openapi.yaml
      - run: pnpm --filter api-types generate
      - run: git diff --exit-code packages/api-types
```

- `concurrency` annule le run précédent de la même branche
- Le cache pnpm évite de tout retélécharger
- `git diff --exit-code` échoue si les **types générés** ne correspondent plus au contrat : quelqu'un a modifié l'un sans l'autre

`ci.yml` (2/2) : chaque côté, en parallèle

```yaml
web:
  needs: contract
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v5
    - uses: pnpm/action-setup@v4
    - uses: actions/setup-node@v5
      with: { node-version: 24, cache: pnpm }
    - run: pnpm install --frozen-lockfile
    - run: pnpm --filter web typecheck
    - run: pnpm --filter web test
    - run: pnpm --filter web build
api:
  needs: contract
  runs-on: ubuntu-latest
  steps: # mêmes étapes, filtre « api »
```

- `needs: contract` : pas de build si le contrat est cassé
- `web` et `api` tournent **en même temps**
- Trois statuts distincts : on sait **quel côté** a cassé
- Les étapes communes peuvent devenir une action composite locale

Détecter une rupture de contrat avant la fusion

```yaml
breaking:
  if: github.event_name == 'pull_request'
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v5
      with: { fetch-depth: 0 }
    - run: git show origin/main:contracts/openapi.yaml > /tmp/base.yaml
    - uses: oasdiff/oasdiff-action/breaking@main
      with:
        base: /tmp/base.yaml
        revision: contracts/openapi.yaml
        fail-on: ERR
```

- `oasdiff` compare deux versions du contrat
- Champ requis ajouté, champ supprimé, type changé : **erreur**
- Une rupture voulue passe quand même : par une PR explicite, avec `!` et un ADR

Protéger `main`

- **PR obligatoire** : personne ne pousse sur `main`
- **Statuts requis** : `contract`, `web`, `api` verts
- **Une revue** au moins, et celle des **propriétaires** (`CODEOWNERS`)
- **Historique linéaire** : fusion en *squash*
- **Pas de force-push**, pas de suppression

Le piège classique

Exiger un statut qui n’existe pas encore : la PR reste bloquée en « Expected — Waiting for status ».

Faites tourner le workflow une première fois, **puis** déclarez ses jobs comme requis dans les règles du dépôt.

API Diplomacy

Niveaux 1 à 3 · Binôme front/back · temps réel

01 Rejoins ton binôme (affiché au tableau). Un **Front**, un **Back** ; trio si besoin.

02 Chacun reçoit un **brief** que l'autre ne voit pas : maquette d'un côté, règles métier de l'autre.

03 Construisez le contrat dans l'**éditeur partagé**. Chat limité : **12 messages** par niveau.

04 « Soumettre le contrat » vérifie les exigences des **deux** briefs.

Temps de jeu

40:00

▶ Cliquer pour lancer

Compétence

Négocier une interface à deux

Où jouer

localhost:5173 → hub → API Diplomacy

Projet · 10 h 40 – 12 h 00

Repo et premier CI

01 **Structure** du monorepo : `contracts/`, `apps/web`, `apps/api`, `docs/adr/`.

02 **CODEOWNERS** : le contrat appartient aux deux sous-groupes.

03 **Workflow** : lint du contrat, puis typecheck, tests et build de chaque côté.

04 **Protection** de `main` : PR, statuts requis, une revue.

05 **Première PR** de chaque sous-groupe, relue par l'autre, titre conventionnel.

Livrable de fin de séance

- ☐ Branche `main` protégée : PR obligatoire, une relecture, CI verte.
- ☐ `.github/workflows/ci.yml` : lint OpenAPI, typecheck, tests, build front et back.
- ☐ Une PR fusionnée, relue par l'autre sous-groupe, avec des commits conventionnels.

Rendu : dépôt Git de l'équipe, branche main

Ce qu'on retient

Vu aujourd'hui

- **Trunk-based** et branches courtes : intégrer tôt, cacher l'inachevé derrière un flag
- La **PR de contrat** passe avant le code ; `CODEOWNERS` impose les deux relectures
- **Monorepo** : un changement de contrat est atomique
- **Conventional Commits** : la rupture devient visible, la version se calcule
- **CI** : le contrat d'abord (lint, types générés, rupture), puis chaque côté

À rendre

- [ ] Branche `main` protégée : PR obligatoire, une relecture, CI verte.
- [ ] `.github/workflows/ci.yml` : lint OpenAPI, typecheck, tests, build front et back.
- [ ] Une PR fusionnée, relue par l'autre sous-groupe, avec des commits conventionnels.

Prochaine séance

Mardi 17 novembre 2026

Choisir sa stack

Frameworks, gestion d'état

Stack Duel

Niveaux 1 à 5

Choisir une architecture qui tient